PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC3533 — Computación de Alto Rendimiento — 2026-1

Tarea 2: Path Tracing con OpenMP y Numba

Viernes 24 de Abril de 2026

En grupos de tres personas, desarrollar y entregar el informe respectivo. Los experimentos deben realizarse en al menos dos computadores distintos y los resultados deben compararse entre sí.

Plazo de Entrega : Viernes 15 de Mayo de 2026, 23:59

Renderizar una imagen 3D consiste en determinar el color de cada pixel a partir de una descripción geométrica de la escena: posición y material de los objetos, fuentes de luz y cámara. En esta tarea se estudian dos algoritmos clásicos de síntesis de imágenes que difieren en cómo modelan la propagación de la luz.

La escena se especifica en `scene.txt` y contiene objetos geométricos, fuentes de luz y materiales:

- **Esferas:** definidas por centro (c_x, c_y, c_z) y radio r.
- **Planos:** definidos por su normal y su distancia al origen; en esta tarea forman las paredes de una habitación cerrada.
- **Materiales:** cada objeto tiene color RGB en $[0, 1]^3$, reflectividad (0 = difuso, 1 = espejo perfecto) y brillo especular. El color actúa como multiplicador por canal: en cada rebote difuso del path tracing, el factor acumulado se multiplica componente a componente por el color de la superficie. Un valor de 1 refleja ese canal completamente; 0 lo absorbe por completo.

El número de objetos K determina cuántas intersecciones se calculan por rayo; la reflectividad determina cuántos rebotes se generan. Ambos factores afectan directamente el costo computacional.

Ray Tracing. La luz física viaja desde las fuentes, rebota en las superficies y llega a la cámara. En ray tracing se invierte este recorrido: se trazan rayos *desde* la cámara *hacia* la escena (*backward ray tracing*). Los pasos por pixel son:

- **Rayo primario:** se calcula una dirección que pasa por el centro del pixel (i, j) y el punto focal de la cámara virtual.
- **Intersección:**[1] el rayo se intersecta con todos los objetos de la escena; se registra el impacto más cercano.
- **Iluminación local:**[2] se evalúan tres términos: *ambiental* (constante, simula luz difusa global), *difuso* (depende del ángulo entre la normal de la superficie y la dirección a la luz) y *especular* (brillo en superficies pulidas).
- **Sombras:** por cada fuente de luz se lanza un rayo de sombra desde el punto de impacto; si choca con otro objeto antes de alcanzarla, el punto queda en sombra.
- **Reflexión:** si la superficie es reflectante, se calcula la dirección de reflexión especular y se lanza un rayo secundario. El proceso se repite recursivamente hasta una profundidad máxima D.
- **Supersampling:**[3] para reducir el efecto de bordes dentados en los objetos, se lanzan S^2 rayos por pixel distribuidos en una grilla $S \times S$ y se promedia el resultado.

El trabajo por pixel varía notablemente: un pixel con cadenas de reflexiones puede llegar a profundidad D, mientras que uno de fondo requiere un solo rayo.

Path Tracing. El ray tracing solo considera iluminación directa, lo que produce imágenes poco realistas: zonas de sombra sin luz rebotada, el color de una pared no tiñe los objetos cercanos[4] y las sombras tienen bordes duros. El path tracing corrige esto integrando la luz que llega de **todas** las direcciones del hemisferio mediante muestreo de Monte Carlo:

[1] Requiere resolver una ecuación geométrica por cada objeto: para esferas y planos las fórmulas son simples (cuadrática y lineal respectivamente); otros cuerpos como mallas de triángulos requieren fórmulas más complejas.

[2] A este modelo de iluminación se le llama modelo de Phong.

[3] Al efecto de bordes dentados que esta técnica busca reducir se le llama *aliasing*.

[4] A este efecto se le llama *color bleeding*.

- **Dispersión aleatoria:** al golpear una superficie difusa, en vez de un rayo de sombra determinista se lanza un rayo en una dirección **aleatoria** del hemisferio orientado según la normal de la superficie.
- **Camino completo:** ese rayo puede rebotar en otra superficie, que a su vez genera otro rayo aleatorio, formando un *camino* (path) desde la cámara hasta el fondo o una fuente de luz.
- **Acumulación de color:**[5] se mantiene un factor que acumula el producto del color del material de cada superficie tocada. Como cada valor está en $[0, 1]$, el factor disminuye con cada rebote, modelando la pérdida de intensidad física de la luz. Cuando cae por debajo de un umbral, el camino se termina anticipadamente pues su contribución es despreciable.
- **Promedio de muestras:** se generan N caminos independientes por pixel y se promedian. Por la ley de los grandes números, el promedio converge al valor real; el error cae como $O(1/\sqrt{N})$, de modo que cuadruplicar N reduce el ruido a la mitad.

El costo por pixel crece en $O(N \cdot D \cdot K)$, donde K es el número de objetos. En ambos algoritmos **cada pixel es completamente independiente**: no hay comunicación ni dependencias de datos, lo que hace el problema trivialmente paralelizable. Sin embargo, el trabajo irregular entre pixeles hace que la elección de estrategia de planificación sea relevante.

Código provisto. En el directorio `src/` se entregan los siguientes archivos de referencia:

- `serial.cpp`: ray tracer serial. Implementa `trace_ray()` con los pasos descritos arriba.
- `serial_pt.cpp`: path tracer serial. Implementa `trace_path()` con acumulación de throughput y dispersión aleatoria por pixel.
- `scene.txt`, `scene_many.txt`: descripción de la escena (esferas y sus materiales).

Los alumnos deben implementar las versiones paralelas con OpenMP (`omp_pt.cpp`) y Numba (`numba_pt.py`).

Configuración del entorno. Los experimentos con OpenMP requieren `g++` con soporte OpenMP (disponible por defecto en Linux). Los experimentos con Numba requieren Python 3.12 con `numpy` y `numba`. Se recomienda usar **Miniconda**:

1. Descargar e instalar Miniconda desde `https://docs.conda.io/en/latest/miniconda.html`.
2. Crear un entorno e instalar las librerías necesarias:
```
conda create -n tarea2-hpc python=3.12 -y
conda activate tarea2-hpc
conda install numpy numba matplotlib -y
```
3. Compilar los códigos de referencia:
```
cd src/
make        # compila serial y serial_pt
```

Los experimentos deben realizarse con ambas escenas (`scene.txt` con 4 esferas y `scene_many.txt` con 40 esferas), variando la resolución y el número de muestras en al menos tres configuraciones: (i) 400×300, S=2, N=32; (ii) 800×600, S=4, N=128; (iii) 1600×1200, S=4, N=256.

1. **[100 pts]** *Path Tracing*
   (a) **[10 pts]** Compile y ejecute `serial.cpp` (ray tracing) y `serial_pt.cpp` (path tracing) en las tres configuraciones. Registre el tiempo de ejecución T_s en cada caso. ¿En cuánto crece el tiempo al duplicar la resolución? ¿Y al duplicar N? Justifique.
   (b) **[25 pts]** Paralelice `serial_pt.cpp` con **OpenMP** agregando una directiva `#pragma omp parallel for` sobre el *loop* externo de pixeles. Experimente con distintas estrategias de planificación:

[5] A este factor acumulado se le llama *throughput* en el código provisto.

(I) `schedule(static)` con distintos valores de *chunksize*.

(II) `schedule(dynamic)` con distintos valores de *chunksize*.

(III) `schedule(guided)`.

Para cada estrategia, mida el tiempo con $p \in \{1, 2, 4, 8\}$ threads en la configuración mediana, con ambas escenas (`scene.txt` y `scene_many.txt`). ¿Cuál estrategia resulta más eficiente? ¿Cambia la estrategia óptima según la escena? Explique por qué el trabajo por pixel es irregular y cómo eso afecta la elección de estrategia.

(c) **[20 pts]** Implemente una versión equivalente en Python acelerada con **Numba**. Use `@njit(parallel=True)` con `prange` sobre el *loop* de pixeles.[6] Explique cómo Numba compila el código y en qué difiere de OpenMP.

(d) **[20 pts]** Compare los tiempos de ejecución de OpenMP (mejor estrategia) y Numba con la referencia serial para las tres configuraciones. Grafique $T(p)$ vs p para ambas implementaciones en una misma figura por configuración. Incluya la curva ideal y el tiempo serial como referencia.

(e) **[15 pts]** Calcule y grafique el *speedup* $S(p) = T_s/T(p)$ y la eficiencia $E(p) = S(p)/p$ vs p para ambas implementaciones. Comente: ¿cuál escala mejor? ¿A partir de qué número de threads decae la eficiencia y a qué se debe?

(f) **[10 pts]** Repita los experimentos en un segundo computador. Para cada computador, declare el número de cores físicos y lógicos y la velocidad de reloj del procesador. Compare los resultados: ¿qué diferencias fueron las más notables? Justifique a qué se deben.

Instrucciones de entrega:

- Entregar un documento PDF con sus resultados a través de Canvas.
- El uso de IA está permitido, mientras esté debidamente declarado en el informe.
- Se evaluará la claridad y legibilidad de los gráficos. Es importante identificar títulos, *labels*, leyendas y tamaño de fuente adecuados.

[6] Preste atención al manejo del generador de números aleatorios: Numba requiere usar semillas independientes por hilo.